# MERIT Application Architecture Requirements

Oct 24, 2026 · @Roscoe

## Purpose

MERIT is built as one ASP.NET Core 10 application that follows model-view-controller (MVC) on the server, serves an Angular client for data-heavy screens, and uses htmx with server-rendered Razor views wherever that takes less code. These requirements add to the MERIT Specification and replace its Front end and Stack choices where the two differ; everything else in the specification, including the data model, row-level security, USWDS styling and WCAG 2.1 AA, still applies.

"Must" marks a requirement; "should" marks a default that a team may depart from with a recorded reason.

## Requirements summary

| ID | Requirement | Verified by |
|---|---|---|
| R1 | The application must follow model-view-controller on the server and in the Angular client: controllers handle requests and user intent, models carry data and rules, views only display. | Architecture tests on every build; code review checklist |
| R2 | Screens assigned to Angular must be written in Angular with TypeScript in strict mode, on an Angular major version that is in active support. | Build settings; dependency check |
| R3 | The server must be one ASP.NET Core application on .NET 10, written in C#, using MVC controllers with Razor views for server-rendered screens and API controllers for the JSON API. | Project files; endpoint inventory test |

| ID | Requirement | Verified by |
|---|---|---|
| R4 | Where a screen needs less code and fewer moving parts as server-rendered HTML than as an Angular screen, it must use Razor views with htmx 4.0 or later, chosen by the rule in "Choosing Angular or htmx". | Screen assignment table, reviewed at each design review |
| R5 | MERIT must ship as one codebase deployed as two programs: a web application that serves users and the API, and a worker that runs background jobs. Both must use the same application, domain and data access code. | Solution layout; deployment manifests; architecture tests |

## Technology baseline

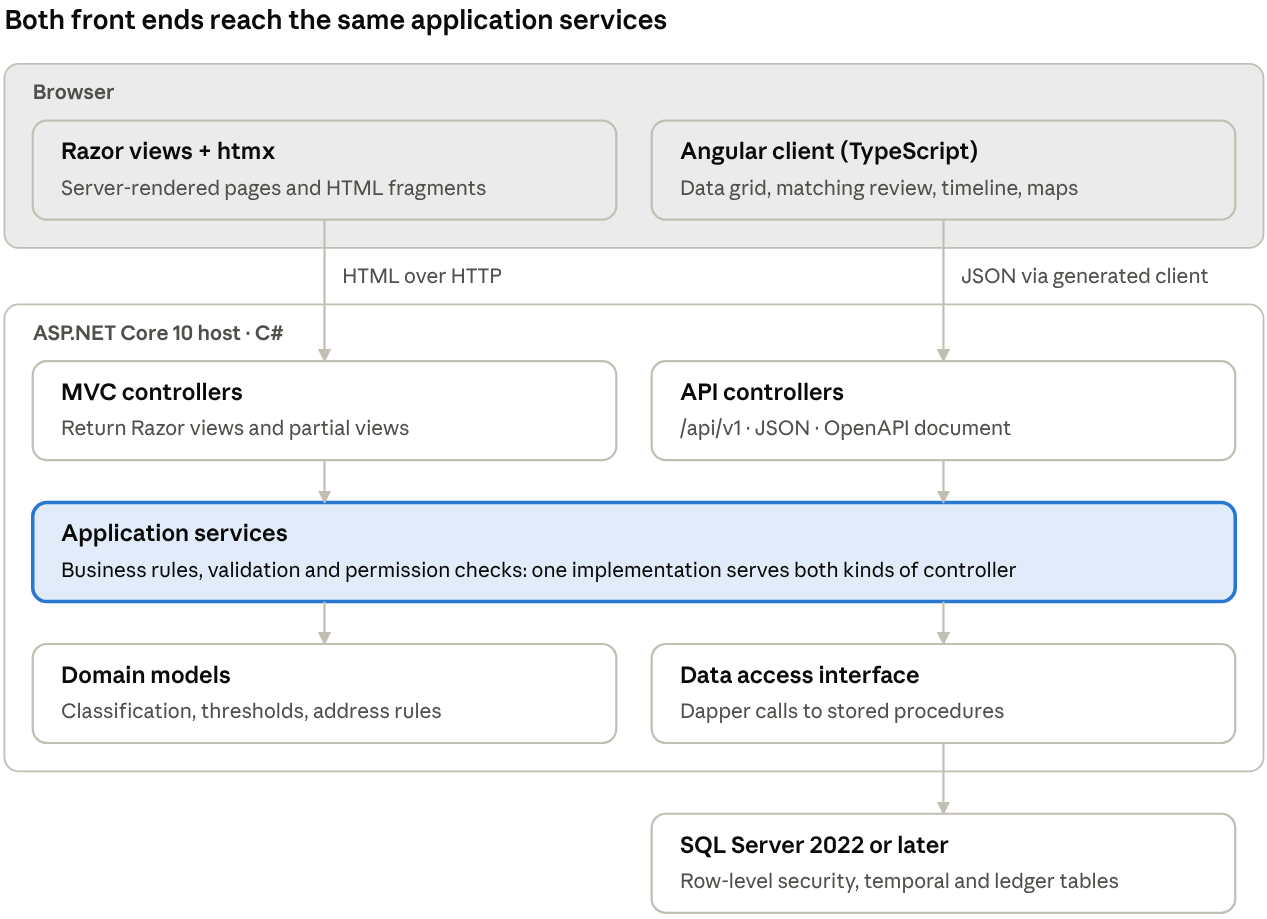
.NET 10 is the long-term support release until November 14, 2028; Angular and htmx are pinned to exact versions and upgraded on the schedule below.

| Component | Required version | Support or status | Upgrade rule |
|---|---|---|---|
| .NET and ASP.NET Core | .NET 10 (LTS), latest patch; 10.0.13 as of October 13, 2026 | Released November 11, 2025; supported until November 14, 2028 | Apply monthly patches within 30 days. Move to the next LTS release before .NET 10 support ends. |
| C# | The language version that ships with the .NET 10 SDK | Follows .NET 10 | No preview language features. |
| Angular with TypeScript | Angular 22, with the TypeScript version its CLI requires | Angular 22 released June 3, 2026; reported active support to June 2027 and LTS to June 2028 | Adopt a new major within 6 months of its release; never ship on a major past LTS. Confirm dates against Angular's own release table. |

| Component | Required version | Support or status | Upgrade rule |
|---|---|---|---|
| htmx | 4.0.x or a later 4.x, exact version pinned | 4.0.0 released August 28, 2026; npm keeps 2.x as `latest` and 4.x under `next` until early 2027 | Install by exact version, never by tag. Project dependency policy allows the pinned 4.0 release while npm tags it `next`. Review each 4.x release notes before upgrading. |
| USWDS | 3.x CSS, fonts and icons, no USWDS JavaScript | As in the main specification | Unchanged |
| SQL Server | 2022 or later | As in the main specification | Unchanged |

## Model-view-controller on the server

Every request goes through a thin controller to an application service, so a rule is written once and both front ends get the same answer.




MERIT layers · two front ends, one service layer

Razor views with htmx call MVC controllers and get HTML back; the Angular client calls API controllers and gets JSON. Both kinds of controller call the same application services.

## Solution layout

| Project | Holds | May reference |
|---|---|---|
| Merit.Web | MVC controllers, API controllers, Razor views and partials, view models, Angular host view, static files | Application, Contracts |
| Merit.Contracts | API request and response types that define the OpenAPI document | Nothing |
| Merit.Application | Application services, validation, permission checks, mapping to view models and contracts | Domain, data access interface |

| Project | Holds | May reference |
|---|---|---|
| Merit.Domain | Domain models and rules: classification, thresholds, address and primary residence rules | Nothing |
| Merit.Data.SqlServer | Data access implementation: Dapper calls to stored procedures | Application's data access interface |

**Rules**

1. **Controllers are thin.** An action binds input, checks the permission policy, calls one application service and picks a result. No business rules, SQL or mapping logic in controllers.
2. **Three kinds of model, never mixed.** Domain models stay inside the application layer. Razor views receive view models built for them. API endpoints accept and return contract types. A domain model is never passed to a view or serialized to JSON.
3. **Views only display.** Razor views and partials use tag helpers and USWDS markup, read only their view model, and never call services or the database.
4. **One rule, one place.** A rule that both front ends need, such as the row split when the primary residence changes, lives in a domain model or application service and is called by both controller kinds.
5. **Controllers, not Razor Pages or Blazor.** Server-rendered screens use MVC controllers and views so the codebase has one pattern. API endpoints use controllers marked `[ApiController]`, not minimal APIs.
6. **Layer rules are tested.** Architecture tests fail the build when a controller references the data project, a view model references a domain type, or the domain project references anything.

## Model-view-controller in the Angular client

Angular is organized around components, so MERIT maps each MVC role onto a fixed Angular construct and keeps the roles in separate files.

| MVC role | Angular construct | Rules |
|---|---|---|
| Model | Contract types generated from the OpenAPI document, plus a feature store per feature that holds state in signals | Generated types are never edited by hand. Stores hold no HTTP or DOM code. |
| View | Presentational components and their templates | Receive data through inputs and report actions through outputs. No injected services, no HTTP, no business rules in templates. USWDS markup and classes. |
| Controller | Route (container) components and feature services | Turn user intent into calls on the generated API client, update the store, handle errors and navigation. One route component per screen. |

- Business rules stay on the server. The client may repeat a rule only to give instant feedback, such as a required field, and the server checks it again.
- Components use standalone APIs, signals and OnPush change detection; features are lazy-loaded routes.
- Lint rules fail the build when a presentational component injects a service or when any file outside the API client calls `HttpClient` directly.

## Choosing Angular or htmx

A screen is built with Razor views and htmx by default, and moves to Angular only when it meets one of the tests below. Most MERIT screens are forms and read-mostly pages whose logic already lives on the server, and htmx serves those with a controller action and a partial view instead of a component, a store and an API endpoint.

**A screen uses Angular when it needs any of these:**

1. The shared data grid: server-side paging, sorting and filtering over large lists, saved views, bulk selection, column chooser, virtual scrolling or grid keyboard navigation.
2. Many interactions in a row that must not wait for the server, such as keyboard-driven review that moves through a queue.
3. Client-side drawing that the user manipulates: zoomable maps, interactive timelines, charts with filtering.

4. Complex overlays built on the Angular CDK, such as nested dialogs over a grid.

Otherwise the screen uses Razor and htmx. Each screen belongs to one front end; Angular components are not embedded in htmx pages or the reverse, and moving between the two is an ordinary link.

MERIT does not support offline use on either front end. Every screen needs a live connection; when a request fails for lack of one, the screen shows a USWDS alert and the user retries once the connection returns. Nothing is queued in the browser.

**Screen assignments**

| Screen | Front end | Reason |
|---|---|---|
| Home dashboard and queue counts | htmx | Server-rendered tiles; counts refresh by polling a partial |
| Work queues, held records, results lists, line lists, query output | Angular | Shared data grid with bulk actions |
| Person matching review | Angular | Keyboard-driven review moving through a queue |
| Person timeline, maps, cluster review | Angular | Interactive drawing |
| Reports and analytics with filters | Angular | Interactive charts |
| Person page | htmx | Each section is a partial that reloads after its own edit; the trend chart is server-rendered SVG |
| Set as primary residence dialog | htmx | The row-split and county-change preview is computed by the server rule anyway |
| Result entry | htmx | Cascading drop-downs are a partial per change; the save summary comes from server rules |
| Intake profile builder | htmx | The server parses the sample; each mapping change returns the preview partial |

| Screen | Front end | Reason |
|---|---|---|
| Case and investigation pages, interviews, samples | htmx | Forms and activity logs |
| Threshold, standards and other configuration screens | htmx | Small tables and forms |
| Federal extracts | htmx | Status cards poll a partial while a job runs; files download directly |
| Sign-in and account pages | Razor without htmx | Plain forms |

A team that wants to move a screen to the other front end records the reason against these tests in the Decisions section of the main specification.

## htmx requirements

htmx 4 changed several defaults from htmx 2, so these rules are written against the htmx 4 migration guide and must not be copied from htmx 2 examples.

**Delivery**

- htmx is installed by exact version, bundled with the site's own scripts and served from MERIT's origin. No CDN.
- Only the extensions MERIT uses are loaded, listed in `htmx.config.extensions`.
- `htmx.config.mode` stays `'same-origin'`, its default, so htmx never calls another origin.

**One action, two shapes**

- Every htmx endpoint is an MVC action that returns a full page or a fragment from the same view model. htmx 4 sends `HX-Request-Type: partial` for fragment requests; a shared result helper returns the partial view for that header and the full view with layout otherwise.
- Any URL that htmx pushes into history must render as a full page on its own. htmx 4 no longer caches pages in `localStorage`; going back re-fetches the page from the server.
- Partial views are named `_Name.cshtml`, live beside their full view and carry a stable `id` on their root element.

- Related regions, such as a queue count in the header, update in the same response with `<hx-partial>` instead of a second request.

### Inheritance, swaps and errors

- Attributes are not inherited in htmx 4 unless marked, such as `hx-confirm:inherited`. MERIT keeps that default and does not set `implicitInheritance`.
- htmx 4 swaps 4xx and 5xx responses by default. MERIT uses that: a failed validation returns status 422 with the form partial re-rendered with its errors, swapped in place.
- A permission failure or server error returns a USWDS alert partial with `HX-Retarget` pointing at the page's alert region. An expired session returns `HX-Redirect` to sign-in.
- Lists that refresh in place use the `innerMorph` swap, so focus and scroll position survive the refresh.

### Scripts and CSP

- No `hx-on` attributes and no `js:` values; behavior that needs script lives in MERIT's own script files.
- htmx runs under the Content Security Policy in the security section, using the `hx-csp` extension the htmx 4 documentation provides for strict policies.
- The `hx-history-cache` extension is not used, because it keeps page copies, which can hold health data, in `sessionStorage`.

### Accessibility

- After a swap that replaces the main content, focus moves to the new content's heading. Smaller updates are announced through one `aria-live` region per page.
- A region waiting on a request sets `aria-busy="true"`, and the request indicator is visible text, not only a spinner.
- Every htmx control is a real link, button or form control, so it works from the keyboard without extra code.

### Dates and type-ahead

- Date fields on htmx screens use the native `<input type="date">`, styled with USWDS input classes. The server validates every date again, because browsers differ in what they accept.
- Searchable choices, such as a provider, organization or LOINC code, use server-side type-ahead: a text input whose `hx-get` returns a USWDS-styled list of matches after the user pauses typing. The list follows the ARIA combobox pattern, works with the keyboard, and the chosen value posts as a hidden ID.

- Neither control adds a JavaScript widget library.

## Angular requirements

The Angular client is one application that owns the screens assigned to it and nothing else.

- **Language and checks.** TypeScript `strict` and Angular `strictTemplates` are on. The lint configuration forbids `any` and fails the build on warnings.
- **Hosting.** Angular screens live under `/app/`. ASP.NET Core serves a Razor host view for those routes that renders the shared header and navigation, the antiforgery cookie and the CSP nonce, then bootstraps Angular into the main region. Users see one header whichever front end draws the page.
- **API client.** The TypeScript client is generated from the OpenAPI document during the build, as the main specification requires, and the build fails if they disagree.
- **Behavior and styling.** Angular CDK supplies overlays, focus management, the table and virtual scrolling for the shared data grid. USWDS supplies CSS, tokens and markup; USWDS JavaScript is not loaded.
- **No health data in the browser.** Nothing from the API is written to `localStorage`, `sessionStorage` or IndexedDB. State lives in memory and is gone when the tab closes.
- **Size budget.** Each lazy-loaded feature has a bundle budget in the CLI configuration, and the build fails when it is exceeded.

## ASP.NET Core host: MVC and Web API in one application

One ASP.NET Core 10 application, registered with `AddControllersWithViews`, serves the Razor and htmx screens, the Angular host view and its files, and the JSON API, all from the same origin.

| Concern | MVC controllers (Razor and htmx) | API controllers |
|---|---|---|
| Routes | Readable page routes, such as `/people/{id}` and `/cases/{id}/investigations` | `/api/v1/...`, versioned in the path |
| Returns | Full views or partial views, with HTTP status codes htmx acts on | JSON contract types; errors as RFC 9457 problem details |

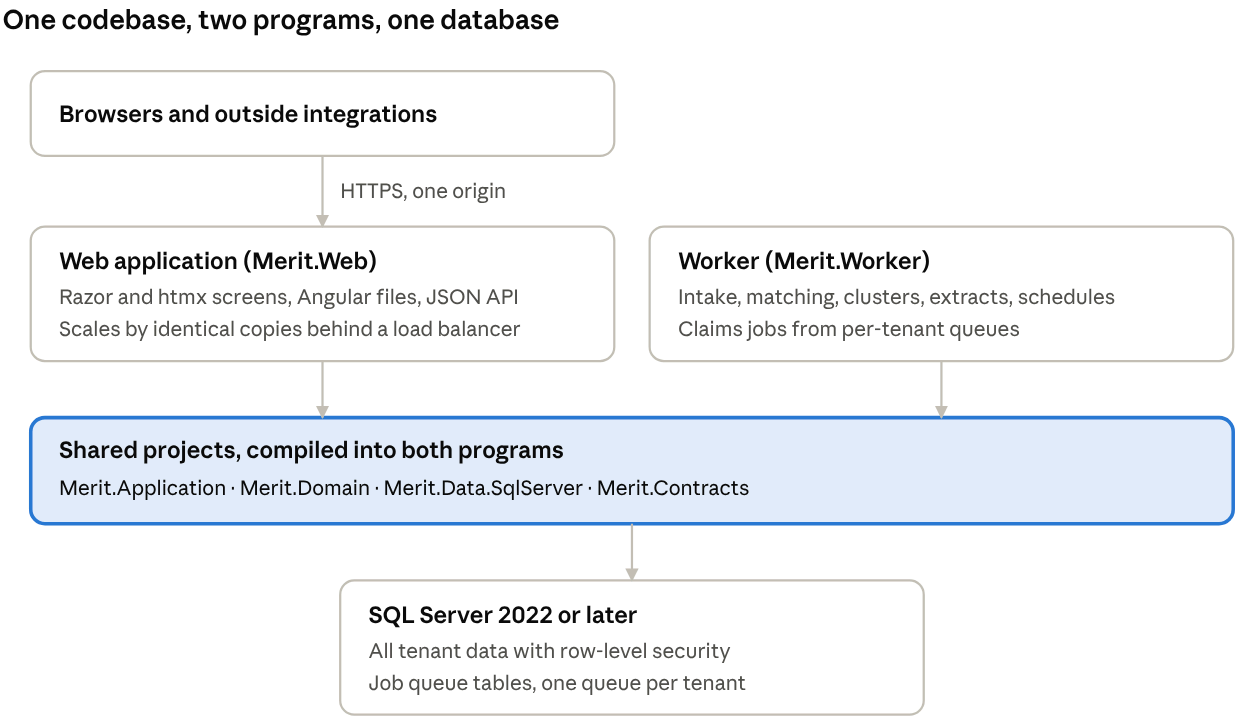
| Concern | MVC controllers (Razor and htmx) | API controllers |
|---|---|---|
| Callers | Browsers | The Angular client and outside integrations |
| Authentication | Session cookie | Session cookie from the browser; a separate token scheme for outside integrations, accepted on API routes only |
| Documentation | Not in OpenAPI | The built-in OpenAPI document, which generates the TypeScript client |

**Shared by both**

- **Request context.** One middleware sets the SQL Server session context for row-level security on every request, as the main specification describes.
- **Authorization.** Every action carries a permission policy, such as `case.edit`. A fallback policy requires a signed-in user, and a test enumerates all endpoints and fails on any action without a policy.
- **Audit, logging and rate limits** run as shared filters and middleware, not per controller.
- **Static files** (the Angular build, htmx, USWDS) are served with fingerprinted names and long cache lifetimes.
- **C# settings.** Nullable reference types on, warnings treated as errors, analyzers on in every project.

## Deployment shape

MERIT is a modular monolith: one repository and one solution, built into two programs that share the same application, domain and data access code. The web application serves people and the API; a worker runs everything that takes longer than a web request.




Deployment shape · web application and worker

The web application records a job and returns at once; the worker claims the job, runs it through the same application services, and writes its result where the screen can poll for it.

| Program | Runs | Scales by | When a copy fails |
|---|---|---|---|
| Web application (`Merit.Web`) | Razor and htmx screens, the Angular host view and files, the JSON API | Identical copies behind a load balancer; no session state kept in memory | Another copy takes the next request |
| Worker (`Merit.Worker`, a .NET 10 worker service) | Intake pipeline from parse to classify, matching runs, cluster detection, federal extract generation, scheduled letters and reports | More copies; each job is claimed by exactly one | The job is retried with back-off, then marked failed on an administration screen; it is never dropped |

**Rules**

1. **Long work runs in the worker.** A request that starts work longer than a few seconds, such as a file upload or an extract run, creates a job and returns. The screen shows the job's status by polling, as the federal extracts screen does.
2. **Queues live in SQL Server.** Jobs should be rows in queue tables, one queue per tenant with the limits the main specification sets, claimed with row locks so two worker copies never take the same job. A separate message broker is added only if measured load needs it.
3. **One copy of every rule.** The worker calls the same application services as the controllers. It never carries its own version of a rule.
4. **Same security.** A job records the tenant and user that started it. The worker sets the SQL Server session context for that tenant before any query, and its actions are audited under that user or a named system account.
5. **Least privilege.** The worker uses its own database login, with EXECUTE only on the procedures its jobs call.
6. **Released together.** Both programs are built from the same commit and deployed together, each as its own container image. Database migrations run once, before either starts. A job written by one release must be readable by the next, so job payloads carry a version number.
7. **Shared web keys.** ASP.NET Core data protection keys are stored where every web copy can read them, so any copy accepts any session cookie.
8. **Health.** Both programs expose health endpoints, and the worker reports queue depth and the oldest waiting job per tenant.

Splitting MERIT into separately deployed services is not planned. It is worth revisiting only if one module needs its own release schedule or its own team.

## Security across both front ends

Both front ends share one session, one antiforgery scheme and one Content Security Policy, so a control is never weaker on one side.

- **Session.** Secure, HTTP-only, `SameSite=Strict` cookies, as in the main specification. No tokens in browser storage.
- **Antiforgery.** ASP.NET Core antiforgery validates every request that changes data, MVC and API alike. Both clients send the token in one header, `X-XSRF-TOKEN`: Angular through its built-in XSRF support reading the `XSRF-TOKEN` cookie, and htmx through a

handler on the `htmx:config:request` event in MERIT's own script. Plain form posts carry the hidden form field.

- **Content Security Policy.** `script-src 'self'` with no `unsafe-inline` or `unsafe-eval`. The Angular host view gives a per-request nonce to Angular through `ngCspNonce`, so component styles load without `unsafe-inline`. htmx follows the script rules in the htmx section.
- **Output encoding.** Razor encodes all output by default. `Html.Raw` is forbidden on any value that came from a user, a file or a lab report, and an analyzer rule enforces it.
- **Same origin only.** No CORS configuration. Outside integrations use the API's token scheme, never the browser session.

## Testing and quality gates

A build fails when any of these checks fails.

| Check | Covers | Tooling |
|---|---|---|
| Unit tests | Domain rules, application services, Angular stores and controllers | xUnit for C#; the Angular CLI's default test runner |
| Integration tests | MVC actions (full page and partial for each htmx endpoint) and API actions, against a real SQL Server | ASP.NET Core `WebApplicationFactory` with a SQL Server container |
| Architecture tests | The layer rules in "Model-view-controller on the server" | An architecture-test library such as NetArchTest |
| Contract check | Generated TypeScript client matches the OpenAPI document | Build step |
| Endpoint inventory | Every action has a permission policy | Integration test |
| End-to-end and accessibility | Key flows on both front ends, keyboard only, at tablet width, with automated WCAG 2.1 AA checks | Playwright with axe |

| Check | Covers | Tooling |
|---|---|---|
| Cross-tenant suite | Unchanged from the main specification | Unchanged |

## Changes to the main specification

| Main specification section | Change |
|---|---|
| Architecture › Stack | Names .NET 10 and C#. Adds ASP.NET Core MVC with Razor views and htmx beside the REST API, which moves under `/api/v1`. |
| Architecture › Front end | Angular renders the screens assigned to it, not every screen. Adds the decision rule and screen assignments from this document. |
| Front end › screen-type table | Forms (person, result entry, case, investigation, interview) move from Angular reactive forms to Razor forms with htmx and server validation. Data-heavy screens stay on the shared Angular data grid. |
| Decisions | Adds this document's choices: MVC on both sides, the htmx-by-default rule, native date inputs and server-side type-ahead on htmx screens, no offline use, and htmx 4 pinned, as project policy allows, while npm still tags it `next`. |
| Architecture (new) | Adds the deployment shape: a modular monolith deployed as a web application and a worker, with per-tenant job queues run by the worker. |

## Sources

- .NET and .NET Core support policy (Microsoft)
- htmx 4.0.0 release announcement
- htmx 2.x to 4.x migration guide
- htmx 4.0.0-alpha announcement

- Angular versioning and releases
- Angular moves to one major release a year